Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

# INF205
# Ressurseffektiv programmering

## 3 Datastrukturar


3.1 Grafdatastrukturar
3.2 Kode-data-samutvikling
3.3 Listedatastrukturar
3.4 Romlege domene
**3.5 Grafalgoritmar**

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

# 3 Datastrukturar

3.1 Grafdatastrukturar

3.2 Kode-data-samutvikling

3.3 Lineære datastrukturar

3.4 Lenkja celledatastrukturar

**3.5 Grafalgoritmar**

INF205

13. mars 2026

# Ordliste

| | |
|---|---|
| connected component | samanhengande komponent |
| priority queue | prioritetskø |
| spanning tree | spenntre |
| sparse graph | glisen graf (tynn graf) |
| subgraph | delgraf |
| subgraph matching problem (subgraph isomorphism problem) | delgrafisomorfiproblem |

# What are typical problems for graphs?

Most important computational problems for graph data structures:

Traversal of the nodes in the graph, including searching. Two canonical ways:

- **Depth first search** (DFS), always goes into depth as far as possible:
  - Push edges to a **stack**; pull from stack to visit the next node.
- **Breadth first search** (BFS), visits nodes in the order they are detected:
  - Push edges to a **queue**; pull from queue to visit the next node.

Reduction to a tree with a given root node (**spanning tree**), for example, using DFS or BFS for a "depth-first" or "breadth-first" tree. Also the **shortest paths**, if nodes have different "distances" from each other (edge weights).

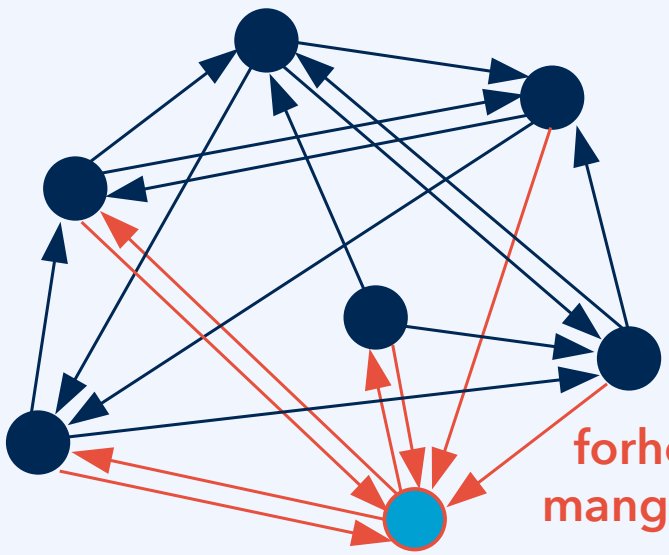
Looking for **paths in a graph**: This includes cycles (same start and end node), the shortest paths (see above), or Hamilton paths/cycles (once at every node).

Related: **Connected components** and strongly connected components.

# Glisne grafar vs. tette grafar

Nabolister (inkl. insidenslister) verkar best hjå *glisne grafar*, som har få kantar. Matrisebaserte datastrukturar er best for *algebraiske formål* og *tette grafar*.

**tette grafar**

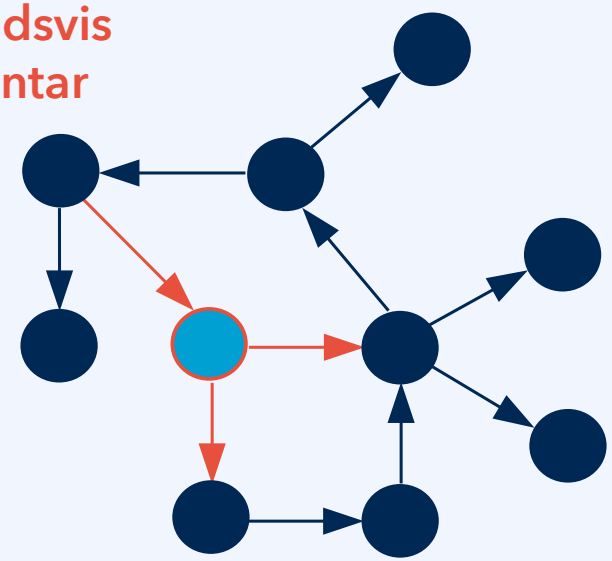


**forholdsvis mange kantar**

$n$ noder,
$e \sim O(n^k)$ kantar

**glisne grafar**
(tynne grafar)

**forholdsvis få kantar**

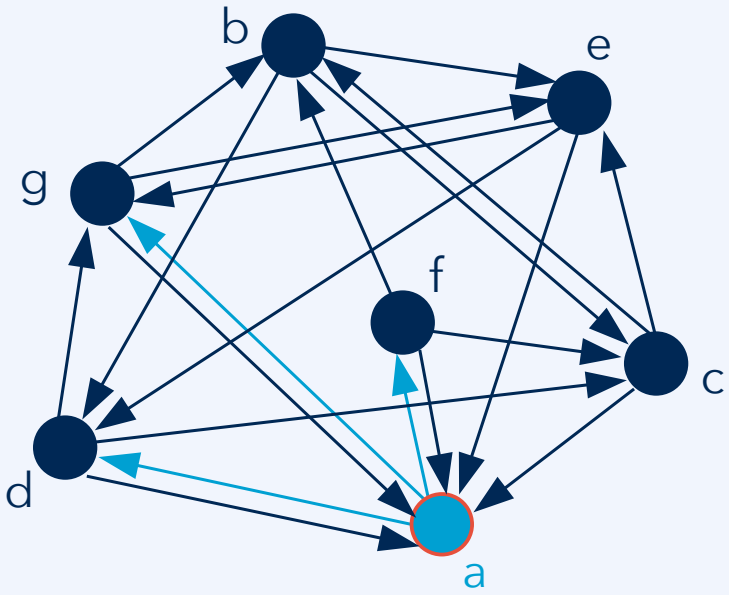


Ofte/strengt: $O(n^2)$ kantar, $k = 2$
Mindre streng definisjon: $k > 1{,}5$

Ofte/strengt: $O(n)$ kantar, $k = 1$
Mindre streng definisjon: $k < 1{,}5$

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt til å besøkje)

**BFS** produserer eit **breidde-fyrst-tre**.

0

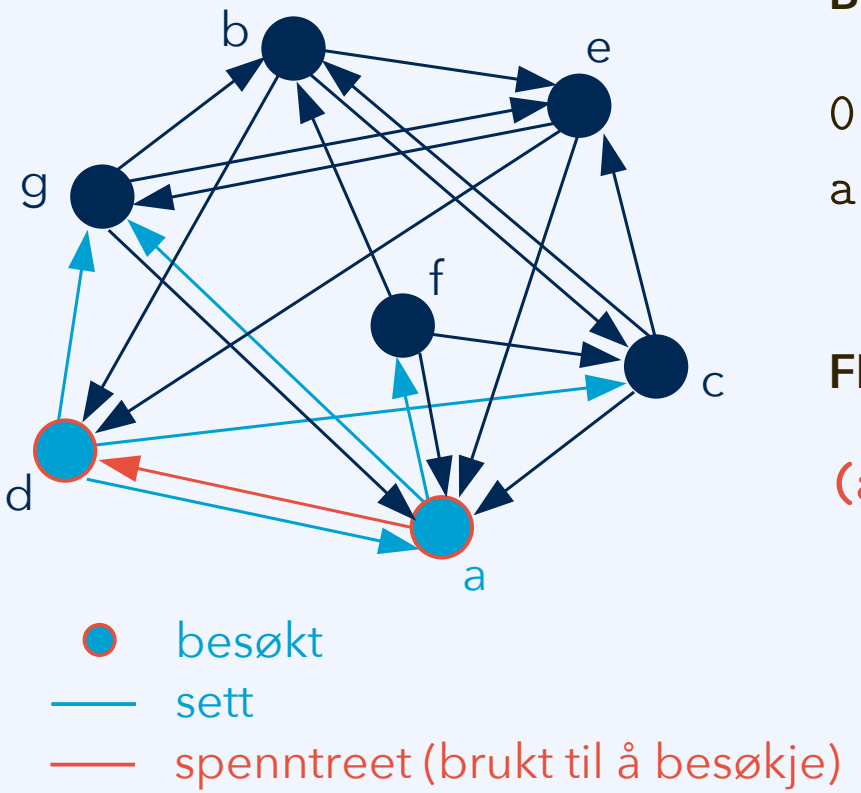
a

**FIFO-kø** anvend på kantar

**(a, d)** (a, f) (a, g)

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




**BFS** produserer eit **breidde-fyrst-tre**.

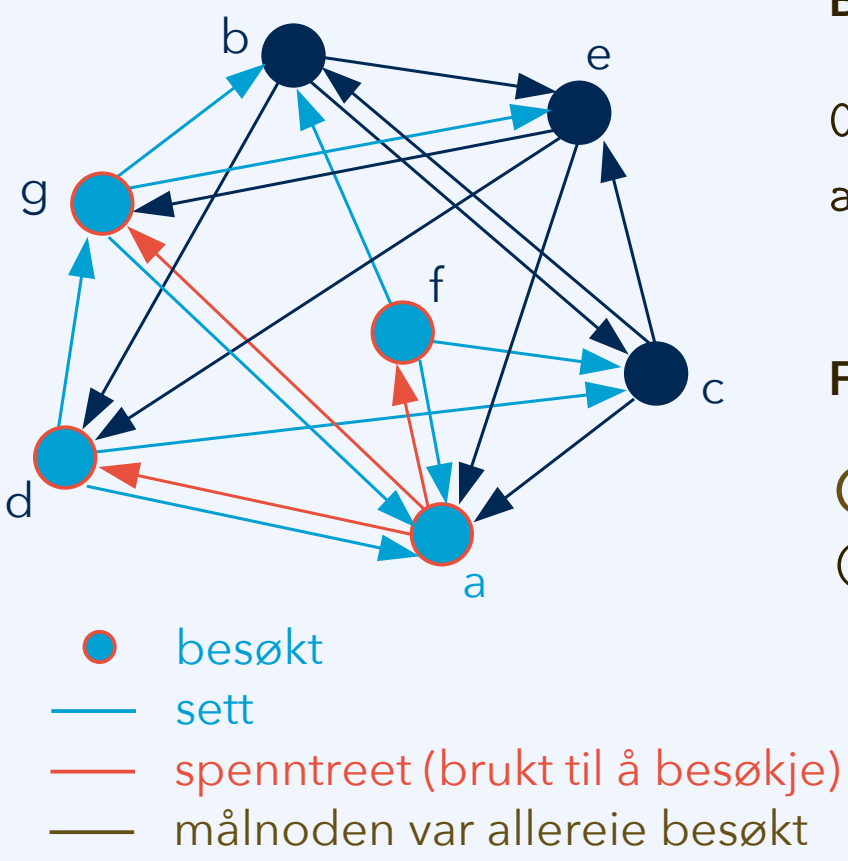
```
0  1
a  d
```

**FIFO-kø** anvend på kantar

```
(a, f) (a, g) (d, a) (d, c) (d, g)
```

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




**BFS** produserer eit **breidde-fyrst-tre**.

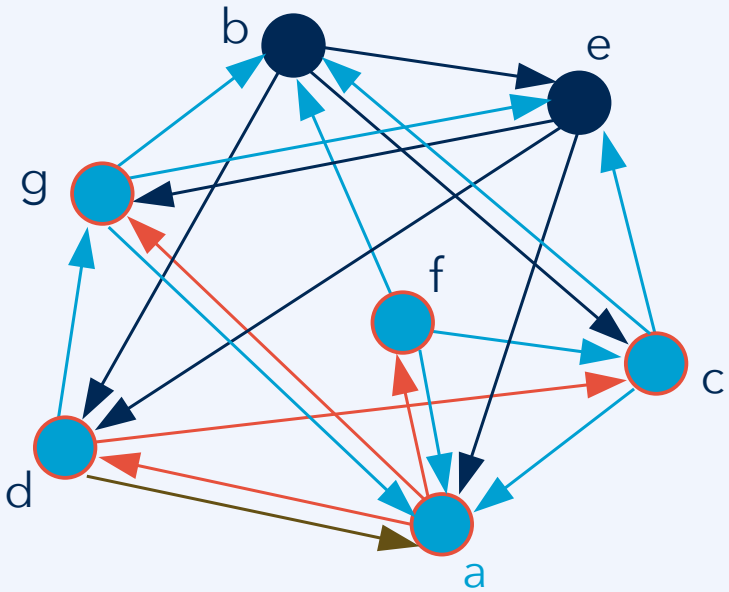
| 0 | 1 | 2 | 3 |
|---|---|---|---|
| a | d | f | g |

**FIFO-kø** anvend på kantar

**(d, a)** **(d, c)** (d, g) (f, a) (f, b) (f, c) (g, a) (g, b) (g, e)

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt til å besøkje)
målnoden var allereie besøkt

**BFS** produserer eit **breidde-fyrst-tre**.

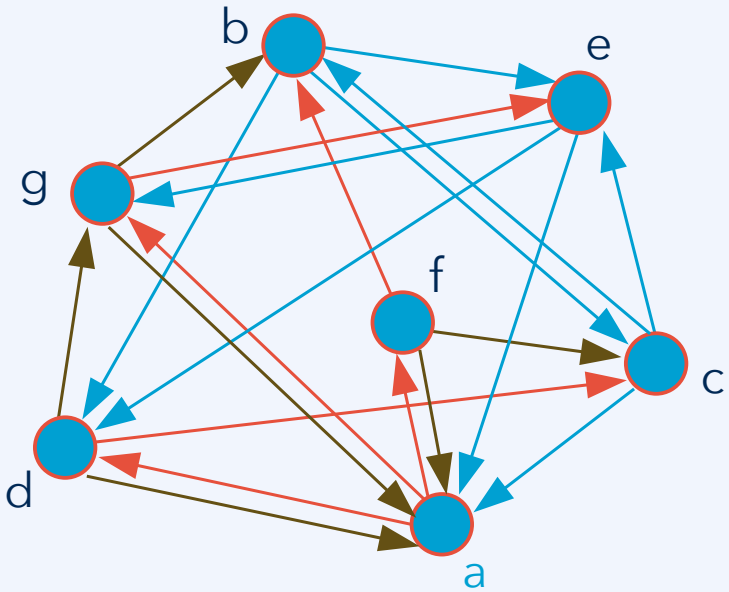
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| a | d | f | g | c |

**FIFO-kø** anvend på kantar

**(d, g)** **(f, a)** **(f, b)** (f, c) (g, a)
(g, b) (g, e) (c, a) (c, b) (c, e)

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt til å besøkje)
målnoden var allereie besøkt

**BFS** produserer eit **breidde-fyrst-tre**.

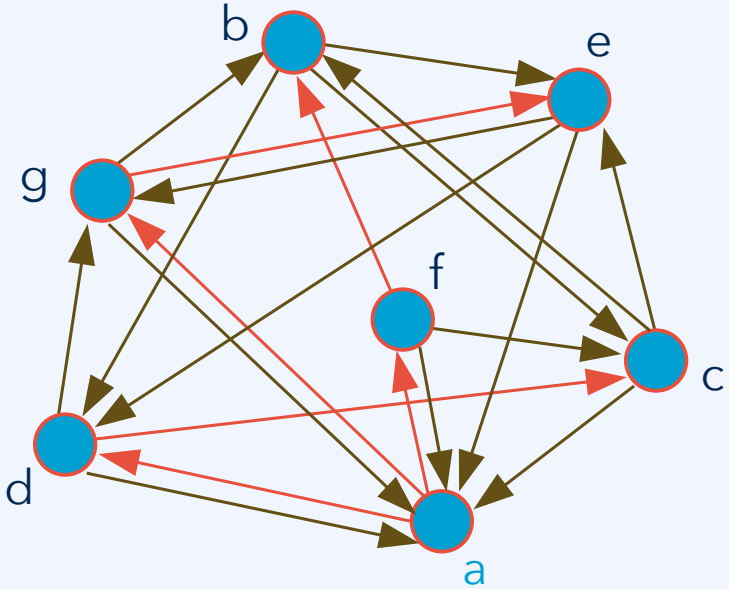
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| a | d | f | g | c | b | e |

**FIFO-kø** anvend på kantar

**(c, a) (c, b) (c, e) (b, c) (b, d) (b, e) (e, a) (e, d) (e, g)**

# Spenntre

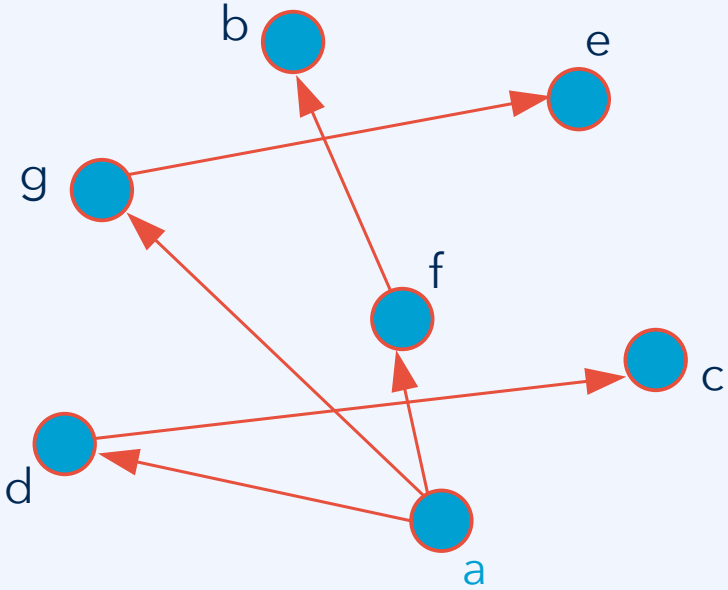
Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt til å besøkje)
målnoden var allereie besøkt

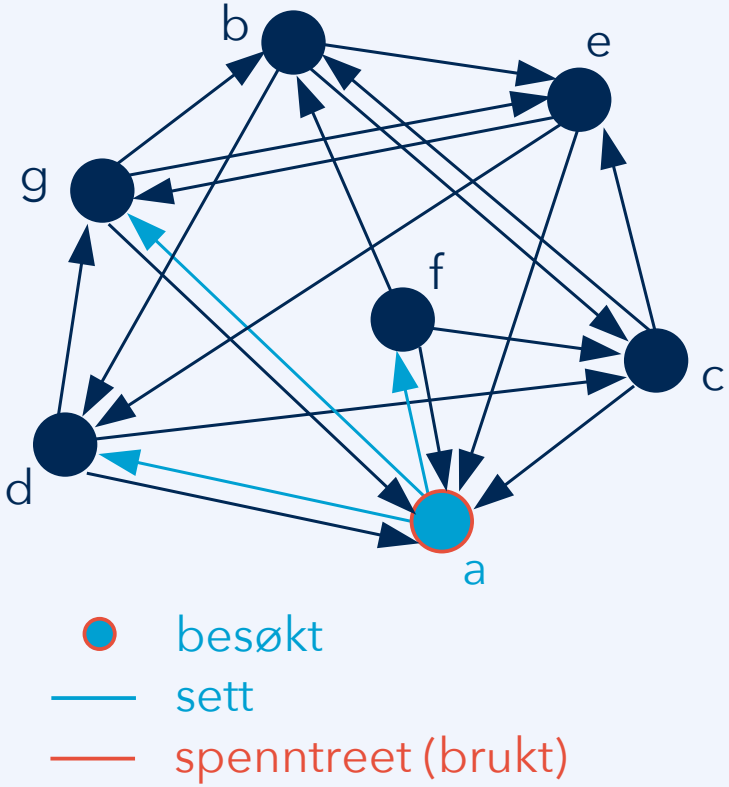
**BFS** produserer eit **breidde-fyrst-tre**.

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| a | d | f | g | c | b | e |



# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




**DFS** produserer eit **djupne-fyrst-tre**.

0

a

**LIFO-stakk** anvend på kantar

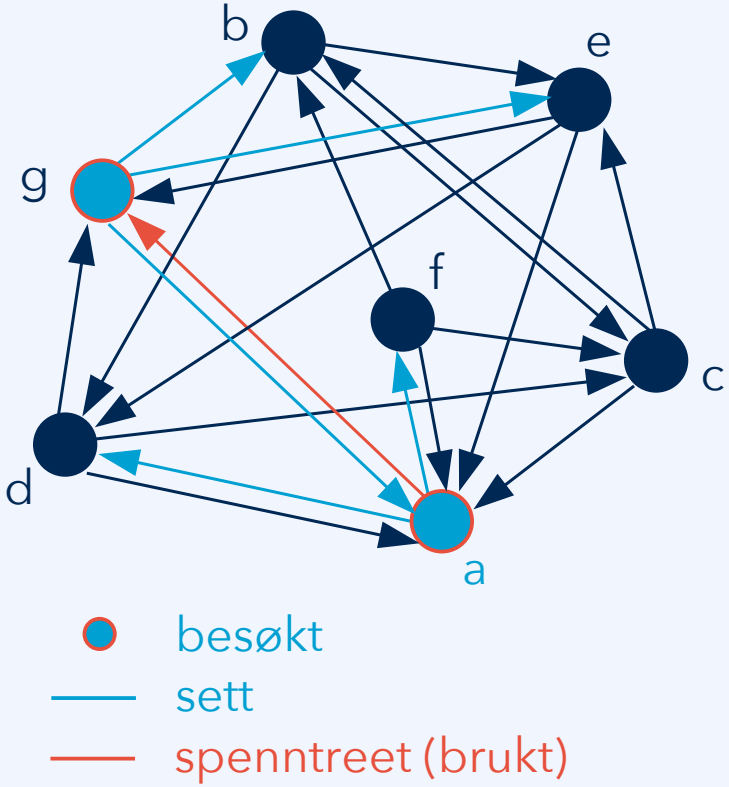
**(a, g)**

(a, f)

(a, d)

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




**DFS** produserer eit **djupne-fyrst-tre**.

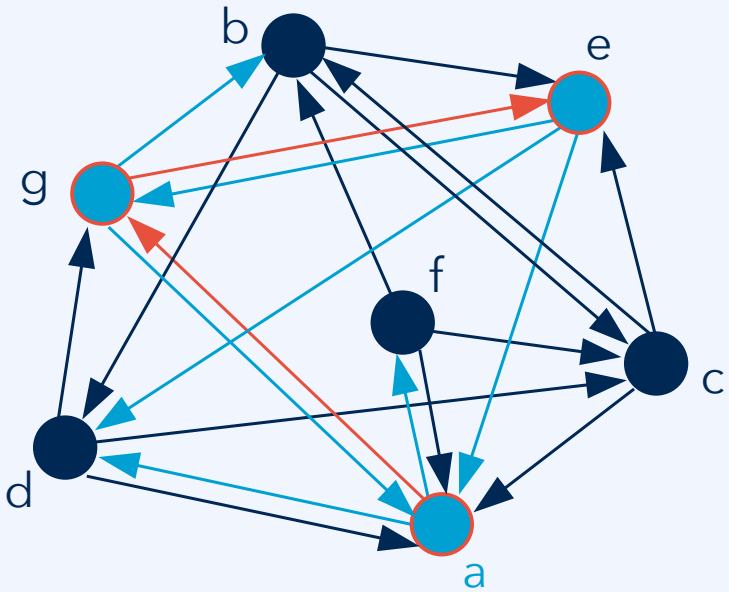
```
0  1
a  g
```

**LIFO-stakk** anvend på kantar

```
(g, e)
(g, b)
(g, a)
(a, f)
(a, d)
```

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt)
målnoden var allereie besøkt

**DFS** produserer eit **djupne-fyrst-tre**.

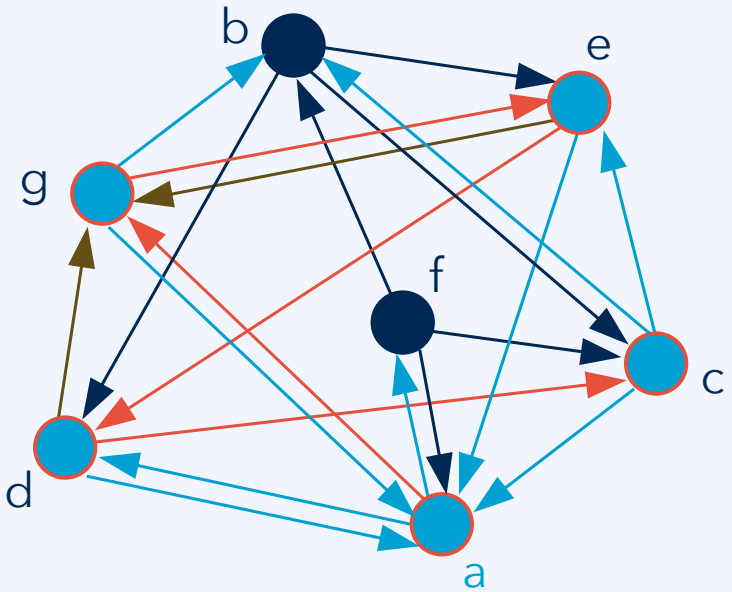
| 0 | 1 | 2 |
|---|---|---|
| a | g | e |

**LIFO-stakk** anvend på kantar

| | | | |
|---|---|---|---|
| | | | (e, g) |
| | | (e, a) | (e, d) |
| | (g, a) | (g, b) | |
| (a, d) | (a, f) | | |

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt)
målnoden allereie besøkt

**DFS** produserer eit **djupne-fyrst-tre**.

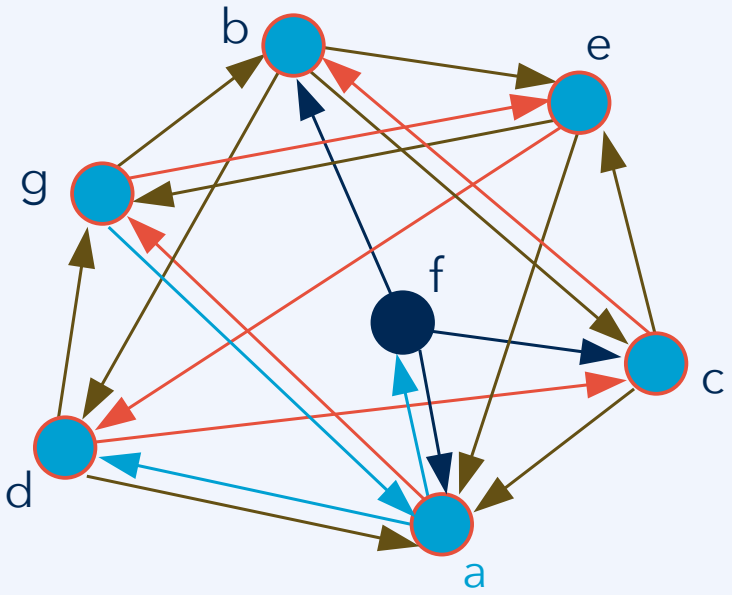
| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| a | g | e | d | c |

**LIFO-stakk** anvend på kantar

```
                                  (c, e)
                        (c, a), (c, b)
              (e, a) (d, a)
        (g, a) (g, b)
(a, d) (a, f)
```

# Spenntre

Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.




besøkt
sett
spenntreet (brukt)
målnoden allereie besøkt

**DFS** produserer eit **djupne-fyrst-tre**.

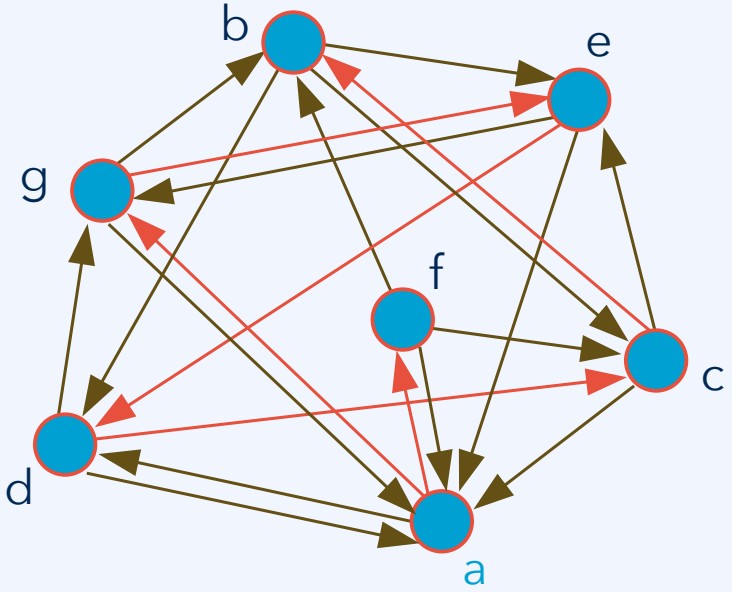
| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| a | g | e | d | c | b |

**LIFO-stakk** anvend på kantar

(g, a)
(a, f)
(a, d)

# Spenntre

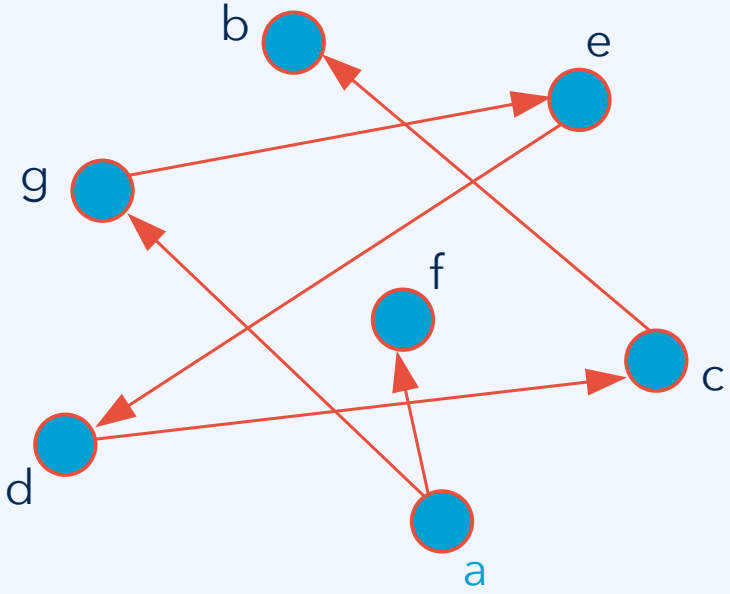
Eit spenntre $T = (V, E')$ for ein graf $G = (V, E)$ er definert gjennom tre kriterium: $T$ må vera eit tre, ein delgraf av $G$, og innehalde heile nodemengda $V$.



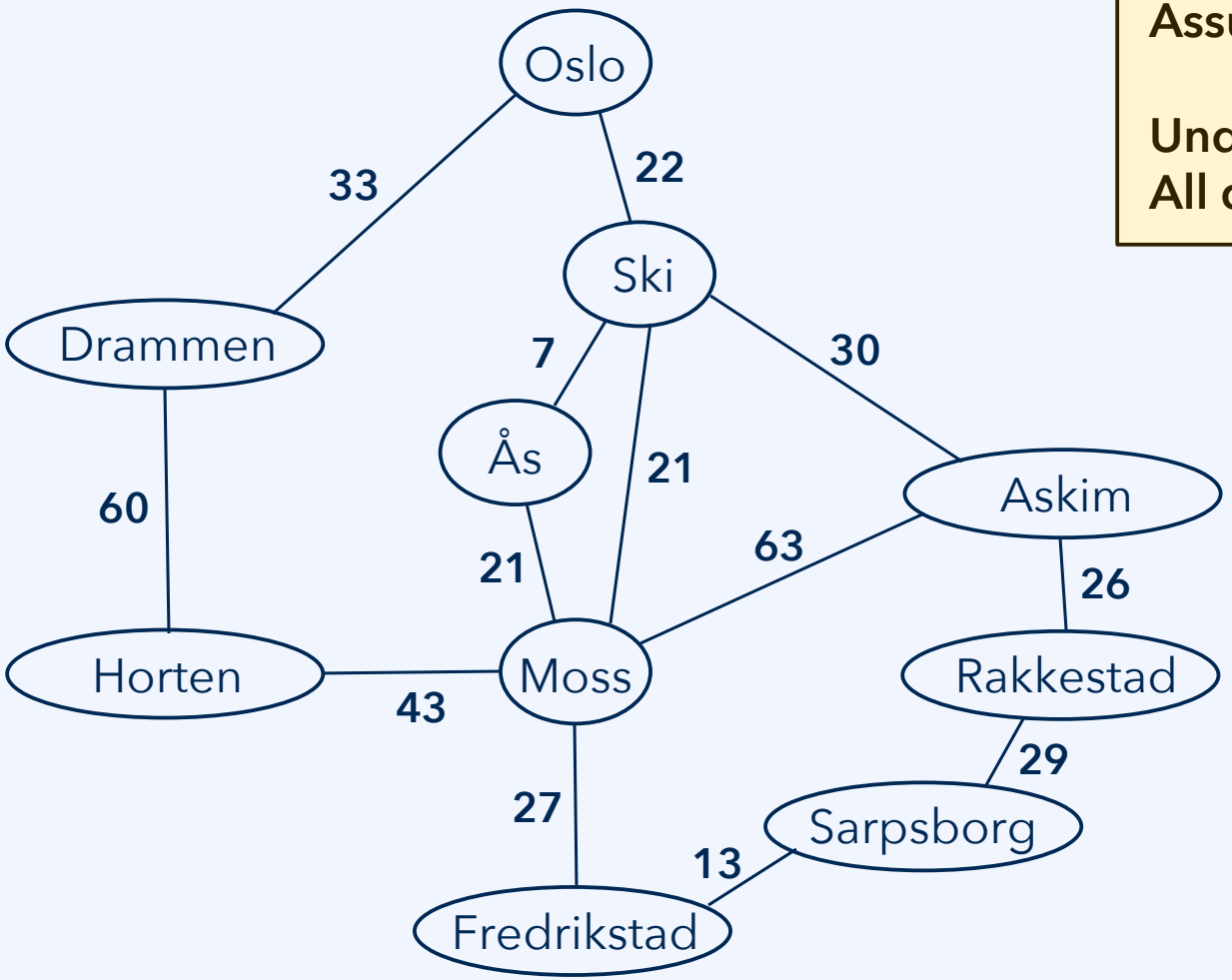

besøkt
sett
spenntreet (brukt)
målnoden allereie besøkt

**DFS** produserer eit **djupne-fyrst-tre**.

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| a | g | e | d | c | b | f |



# Shortest paths: Dijkstra's algorithm




**Assumptions:**

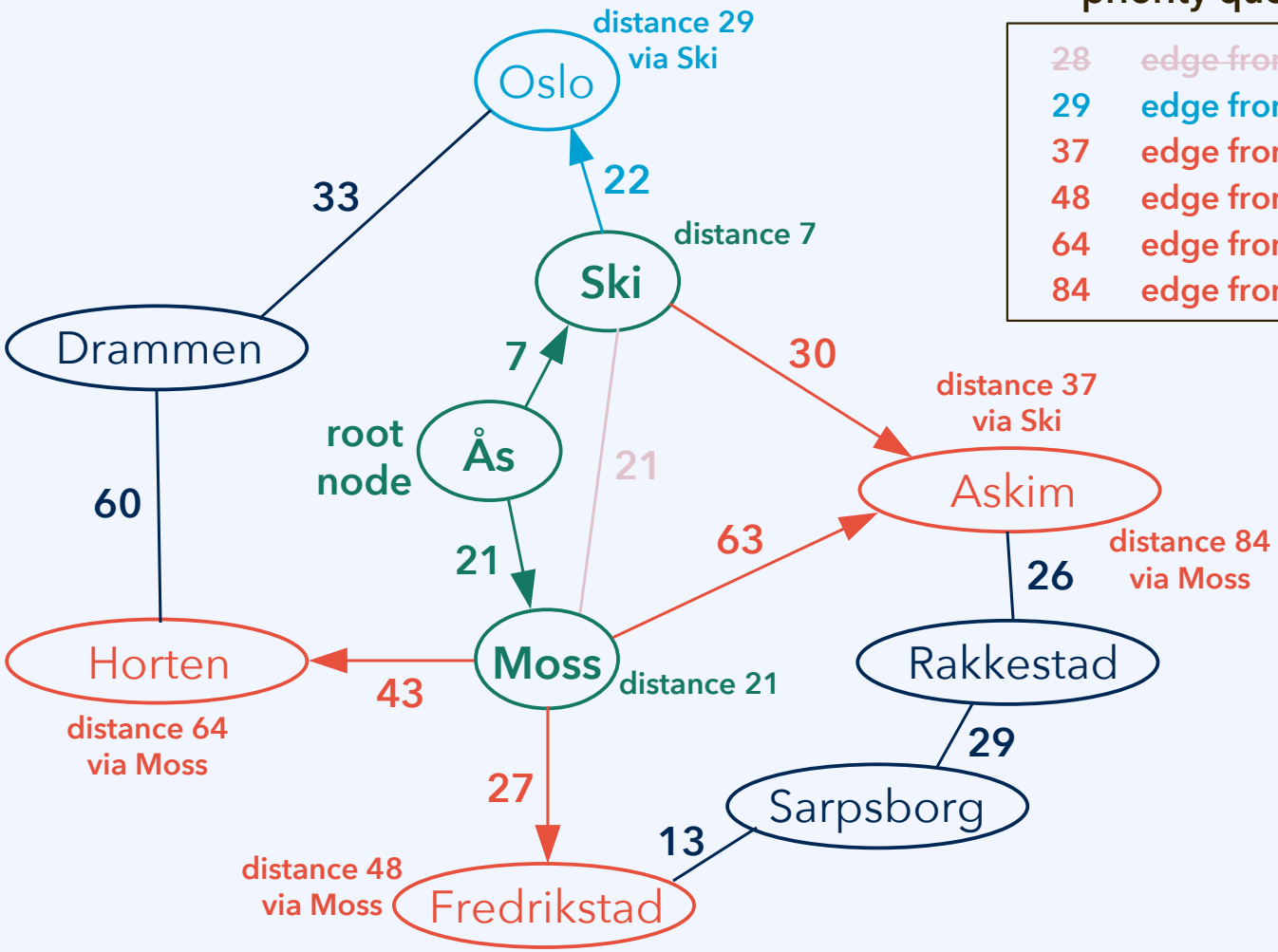
**Undirected graph**
**All distances are positive**

In each iteration, visit the detected node closest to the root.

Process all edges to which that node is incident, detecting any undetected neighbours, and updating tentative distances.

**Example:** incidence-list-graph.zip

# Shortest paths: Dijkstra's algorithm

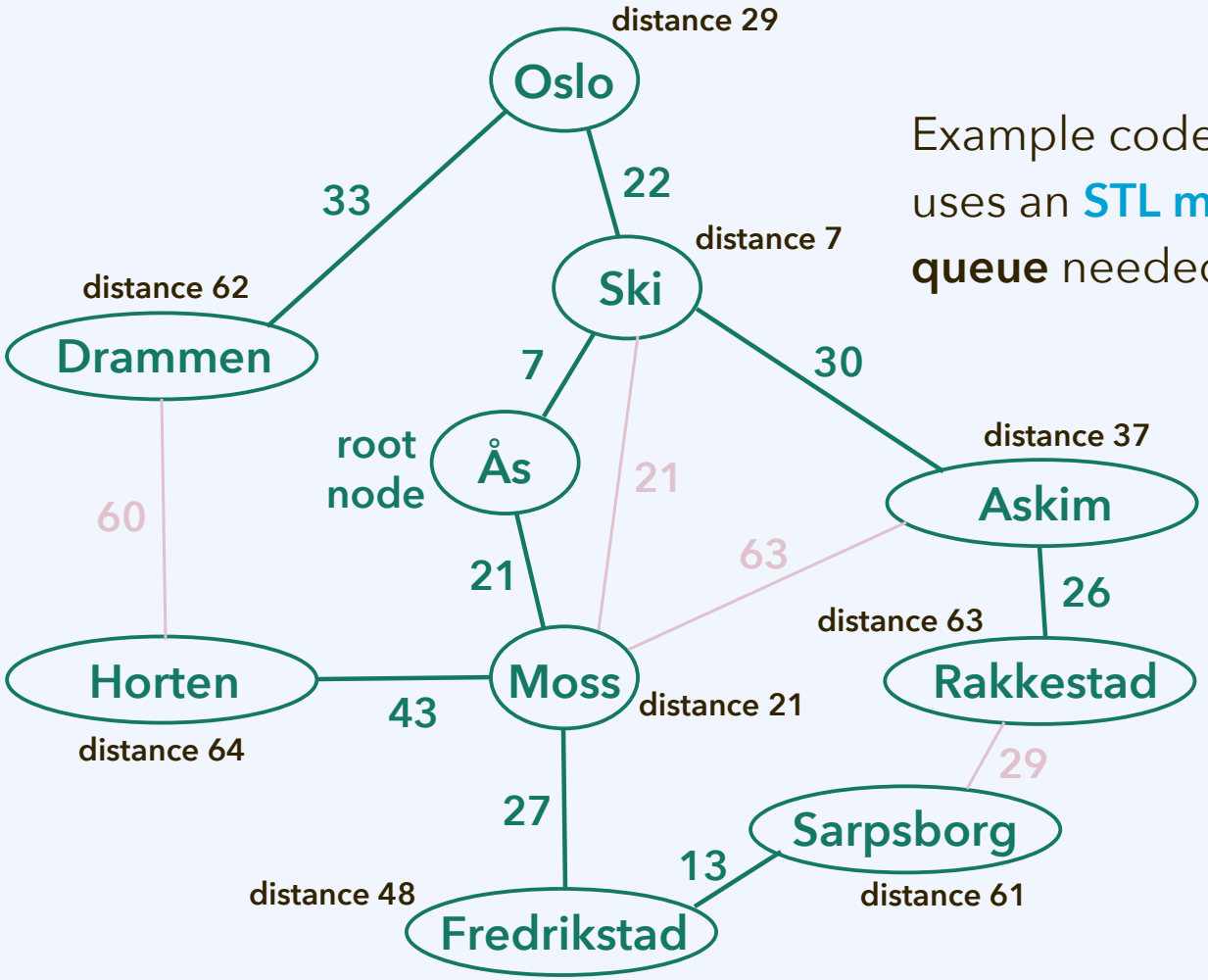



**priority queue** data structure

| | |
|---|---|
| ~~28~~ | ~~edge from Ski to Moss~~ |
| 29 | edge from Ski to Oslo |
| 37 | edge from Ski to Askim |
| 48 | edge from Moss to Fredrikstad |
| 64 | edge from Moss to Horten |
| 84 | edge from Moss to Askim |

**Example:** incidence-list-graph.zip

# Shortest paths: Dijkstra's algorithm

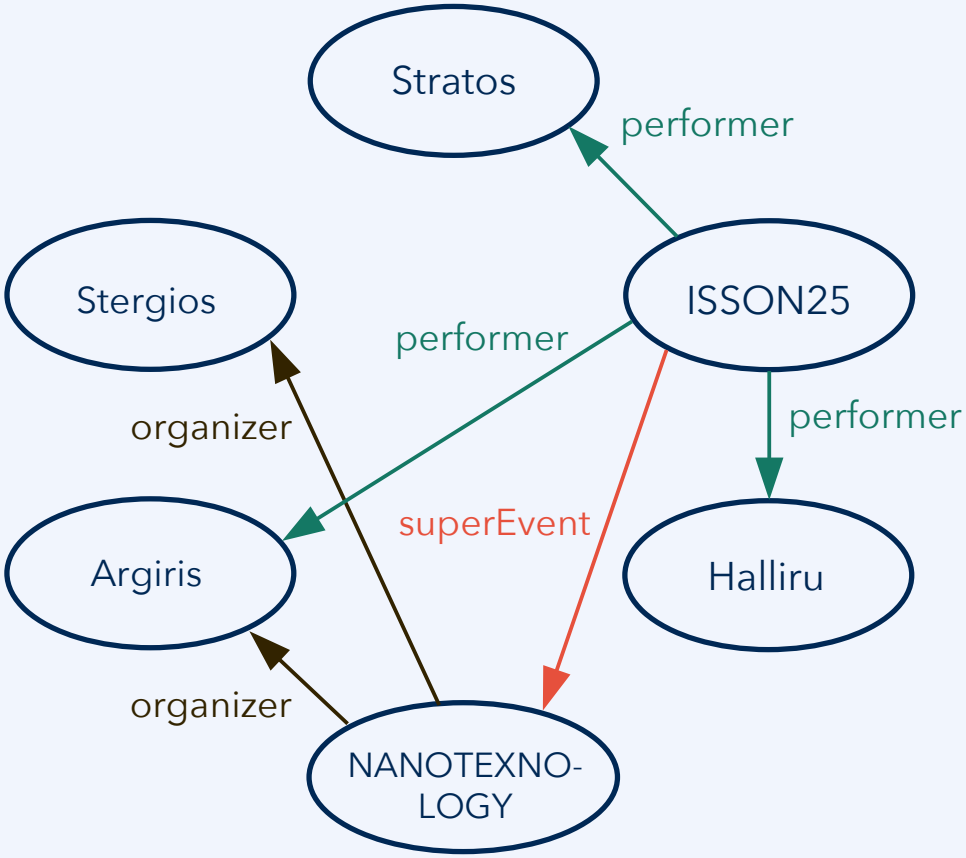



Example code **incidence-list-graph** uses an **STL multimap** as the **priority queue** needed in Dijkstra's algorithm.

A **tailored data structure** is used both for the graph and for the tree that contains all the shortest paths.

**Example:** incidence-list-graph.zip

# Nodehierarki: Sterkt samanhengande komponentar




**hierarkisk struktur (spenntre)**


```
{
  "label": "ISSON25",
  "performer": [
    {
      "label": "Halliru"
    }, {
      "label": "Stratos"
    }, …
  ],
  "superEvent": {
    "label": "NANOTEXNOLOGY",
    "organizer": [
      {
        "identifier": "Argiris"
      }, {
        "identifier": "Stratos"
      }
    ]
  }
}
```

# Sterkt samanhengande komponentar

**Tarjans algoritme**

*Obs: Nodar høyrer til den same komponenten om og berre om det er ein sti frå den eine til den andre, og omvendt!*
Traverser grafen systematisk og slå saman alt som ligg på ein felles syklus.

Implementering: **Djupne-fyrst-søk** med eit attributt i tillegg: **Låglenkja**/lowlink. Det er den lågaste (fyrst besøkte) noden som er blitt nådd frå ein viss node. Algoritmen er basert på bruk av ein **nodestakk**.

- Tildel indeks (ordenstal) til nodane i den rekkjefylgja dei blir besøkt.
- Gå gjennom kantane rekursivt (ekvivalent til å bruke ein kantestakk)
  - Er målnoden enno ikkje besøkt? **Legg på stakk** og besøk (rekursjon). Oppdater låglenkja til aktuell node når rekursjonen returnerer.
  - Hadde vi allereie beskøkt målnoden? Og står den på stakken? Oppdater låglenkja. *(Ikkje oppdater om målnode ikkje er på stakk!)*
- Er nodeindeksen no den same som låglenkja til noden?
  - Fjern **aktuell node og alt ovanfor frå stakken**. Det **er éin komponent**!

# Querying graph databases using SPARQL

SPARQL is a recursive acronym: "SPARQL Protocol and RDF Query Language." An interface that can handle SPARQL queries is called a *SPARQL end point*.

The **syntax of SPARQL** is reminiscent of SQL, but at its core is given by

- **RDF triples**, using RDF schema and OWL, in **TTL notation**;
- Some elements of the triples are **wildcards**, *i.e.*, free variables.

```
SELECT ?x ?y
WHERE {
  ?x erInstansAv ?emnekode.
  ?emnekode tilbyddAvFakultet ?fakultet.
  ?fakultet harStudieveilder ?y.
}
```

"Produce a table with course instances in the first column and the study advisors from the faculty responsible for the course in the second colum."

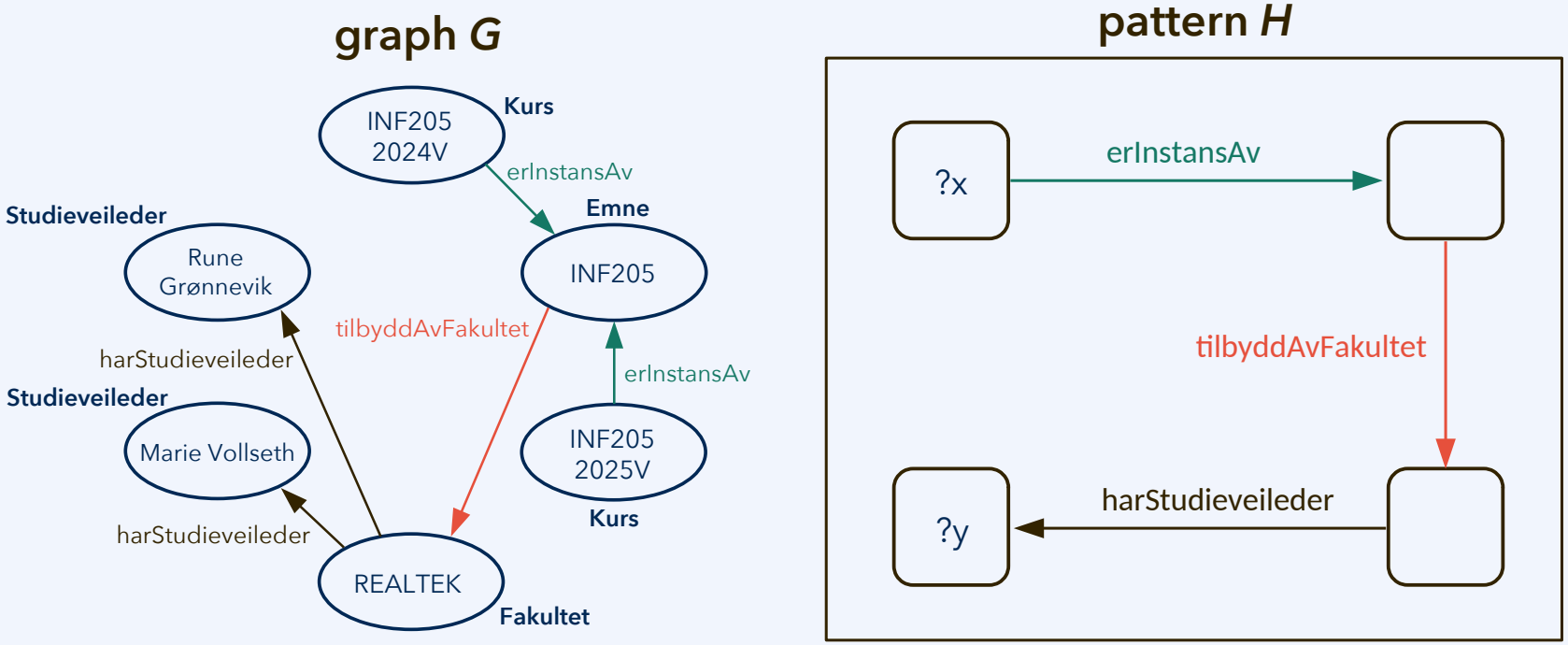
The **semantics of SPARQL** is given by the correct response to a SPARQL query, which consists of a *table with all matching valuations of the selected wildcards*.

SPARQL querying therefore corresponds to the **subgraph matching problem** from graph theory: It looks for occurrences of a pattern within a larger graph.

# Querying graph databases

**Subgraph matching problem** (NP-complete):

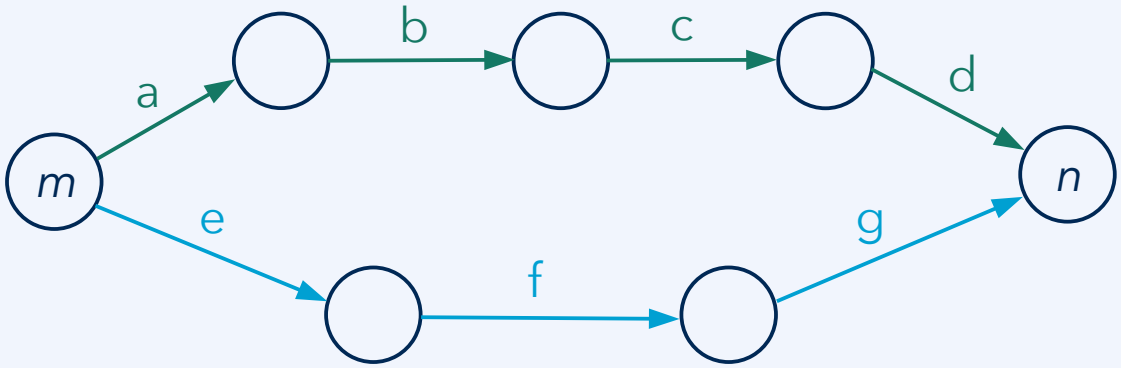
Given a graph *G* and a pattern *H*, does G contain a subgraph isomorphic to *H*?




SPARQL querying therefore corresponds to the **subgraph matching problem** from graph theory: It looks for occurrences of a pattern within a larger graph.

# Suggested topic formulation

Given a graph $g$ and pair of edge-label sequences $(p, q)$, decide whether there is a pair of nodes $(m, n)$ in $g$ such that $m$ is connected to $n$ via a path out of edges labelled $p[0], p[1], \ldots$, <u>and</u> by a path of edges labelled $q[0], q[1], \ldots$




$p = \{a, b, c, d\}$

$q = \{e, f, g\}$

*Are we considering directed or undirected graphs?* For the standard use case, directed graphs are typical. So it could make sense to choose directed graphs.

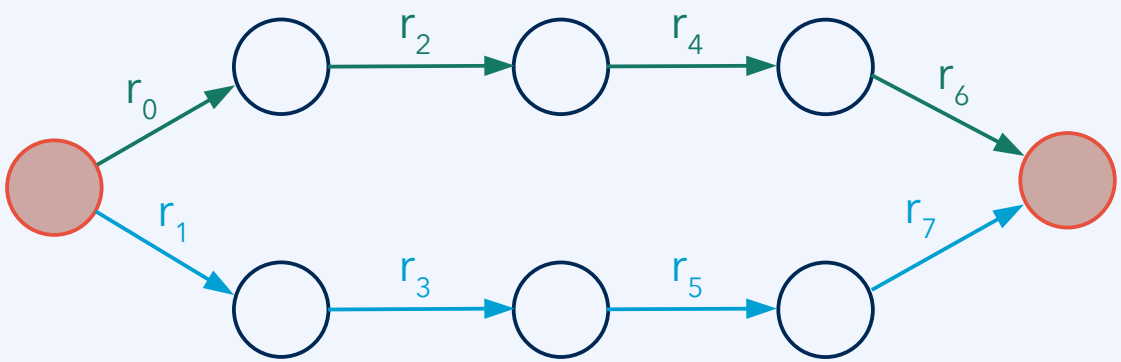
*What if the pattern occurs many times in the graph?* It is formulated as a yes-no question (decision problem). In reality, we could be interested in all matches.

# Benchmark scenario: Graph queries

See **graph-benchmark** generator code. Two main scenario parameters:

- Graph size *n*, the number of nodes in the graph (not number of edges)
- Query size *m*, length of each of the two paths from the query

**Benchmark query pattern** for $m = 4$




**query.dat** output file

```
<r0> <r2> <r4> <r6> <>
<r1> <r3> <r5> <r7> <>
```

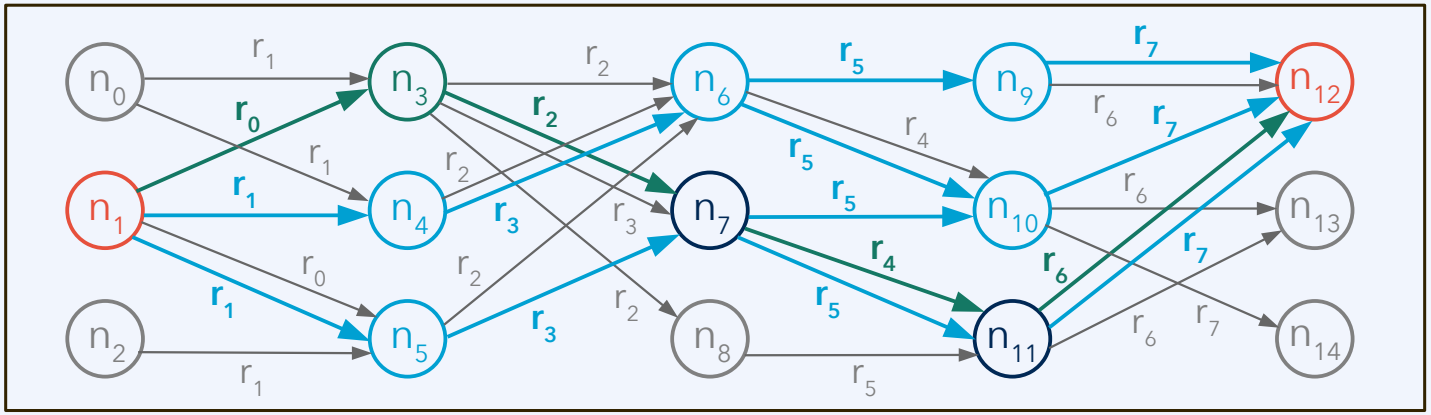
$p = \{r_0, r_2, r_4, r_6\}$

$q = \{r_1, r_3, r_5, r_7\}$

Random graphs are generated such that both cases with and without an instance of the graph pattern occur (the aim was to approximate a 50:50 ratio).

# Benchmark scenario: Graph queries

Random benchmark graph from "./generate-graph 15 4 kb.dat query.dat":




| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| <n0> | <r1> | <n3> | <n3> | <r2> | <n6> | **<n6>** | **<r5>** | **<n9>** | <n9> | <r6> | <n12> | <> |
| <n0> | <r1> | <n4> | **<n3>** | **<r2>** | **<n7>** | <n6> | <r4> | <n10> | **<n9>** | **<r7>** | **<n12>** | |
| **<n1>** | **<r0>** | **<n3>** | <n3> | <r3> | <n7> | **<n6>** | **<r5>** | **<n10>** | **<n10>** | **<r7>** | **<n12>** | |
| **<n1>** | **<r1>** | **<n4>** | <n3> | <r2> | <n8> | **<n7>** | **<r5>** | **<n10>** | <n10> | <r6> | <n13> | |
| <n1> | <r0> | <n5> | <n4> | <r2> | <n6> | **<n7>** | **<r4>** | **<n11>** | <n10> | <r7> | <n14> | |
| **<n1>** | **<r1>** | **<n5>** | **<n4>** | **<r3>** | **<n6>** | **<n7>** | **<r5>** | **<n11>** | **<n11>** | **<r6>** | **<n12>** | |
| <n2> | <r1> | <n5> | <n5> | <r2> | <n6> | <n8> | <r5> | <n11> | **<n11>** | **<r7>** | **<n12>** | |
| | | | **<n5>** | **<r3>** | **<n7>** | | | | <n11> | <r6> | <n13> | |

**kb.dat**
output file

The benchmark graphs consist of $m$+1 partitions.

# Questions and answers

- *Should the two paths have the same starting node?* Yes.
- *Is it the idea that the user provides the start node?* No, we search it.
- *Are the labels unique?* Nodes have unique labels, and edges don't.
- *Can the same node occur in both paths?* Yes.
- *What realistic queries are there that take the form of sequences p and q?*
  - "What inventors were killed by their own invention?"[1]
  - "What products are of the same colour as their packaging?"
  - "What ministers had children who became ministers of the same country?"[1]

[1]See https://www.wikidata.org/wiki/Wikidata:SPARQL_query_service/queries/examples/

# Glossar

*delgraf*

*delgrafisomorfiproblem*

*spenntre*

# Glossar

*delgraf*

Definisjon: graf $G' = (V', E')$ som er del av ein større graf $G = (V, E)$ slik at $V' \subseteq V$ og $E' \subseteq E$

*delgrafisomorfiproblem*

Definisjon: berekningsproblem der to grafar $G$ og $H$ blir gjevne som input, og ein algoritme skal bestemme om $H$ er isomorf til (dvs. har nøyaktig den same strukturen som) ein subgraf av $G$

*Dvs.: Førekjem grafmønsteret H i den større grafen G?*

*spenntre*

Definisjon: graf $T = (V, E')$ som er eit tre, dvs. som er asyklisk og samanhengande, og som er ein delgraf av ein annan graf $G = (V, E)$, $E' \subseteq E$, slik at treet $T$ inneheld alle nodane frå grafen $G$

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

# INF205
# Ressurseffektiv programmering

## 3 Datastrukturar


3.1 Grafdatastrukturar
3.2 Kode-data-samutvikling
3.3 Listedatastrukturar
3.4 Romlege domene
**3.5 Grafalgoritmar**